

Player Pointer Offset Issues



```
#include "stdafx.h"
#include <iostream>
#include "mem.h"

// Created with ReClass.NET 1.2 by KN4CK3R

class ent
{
public:
    char pad_0000[4]; //0x0000
    float X; //0x0004
    float Y; //0x0008
    float Z; //0x000C
    char pad_0010[36]; //0x0010
    float lookleft_right; //0x0034
    float lookup_down; //0x0038
    char pad_003C[176]; //0x003C
    int32_t player_health; //0x00EC
    int32_t armor_quantity; //0x00F0
    char pad_00F4[20]; //0x00F4
    int32_t pistol_stored_ammo; //0x0108
    char pad_010C[16]; //0x010C
    int32_t assault_rifle_stored_ammo; //0x011C
    char pad_0120[32]; //0x0120
    int32_t assault_rifle_ammo; //0x0140
    int32_t grenade_number; //0x0144
    char pad_0148[8]; //0x0148
    int32_t pistol_shot_reload_delay; //0x0150
    char pad_0154[16]; //0x0154
    int64_t assault_rifle_shot_reload_delay; //0x0164
    char pad_016C[8]; //0x016C
    int32_t pistol_total_shots; //0x0174
    char pad_0178[16]; //0x0178
    int32_t assault_rifle_total_shots; //0x0188
    char pad_018C[80]; //0x018C
    int32_t number_of_kills; //0x01DC
    char pad_01E0[32]; //0x01E0
    //char name ? [4]; //0x0200
    //char name1[4]; //0x0204
    //char name2[4]; //0x0208
```

```

char pad_020C[268]; //0x020C
bool is_dead; //0x0318
char pad_0319[75]; //0x0319
class N000002EE* weapon_in_hand; //0x0364
char pad_0368[480]; //0x0368
}; //Size: 0x0548


class N00000291
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N00000291) == 0x44);


class N000002EE
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N000002EE) == 0x44);


//above is for padding


DWORD WINAPI HackThread(HMODULE hModule)
{
    // Create Console
    AllocConsole();
    FILE* f;
    freopen_s(&f, "CONOUT$", "w", stdout);

    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <OFF>\n";
    std::cout << "[F2] Ammo Hack : <OFF>\n";
    std::cout << "[F3] No Recoil : <OFF>\n";
    std::cout << "[F4] No Reload : <OFF>\n";
    std::cout << "[INS] Exit\n";
    std::cout << "=====\\n";


    uintptr_t moduleBase = (uintptr_t)GetModuleHandle(L"ac_client.exe");
    moduleBase = (uintptr_t)GetModuleHandle(NULL);

    bool bHealth = false, bAmmo = false, bRecoil = false, bNoReload = false,
        bGrenade = false;

    while (true)

```

```
{
    if (GetAsyncKeyState(VK_INSERT) & 1)
    {
        break;
    }

    bool updateDisplay = false;

    if (GetAsyncKeyState(VK_F1) & 1)
    {
        bHealth = !bHealth;
        updateDisplay = true;
    }

    if (GetAsyncKeyState(VK_F2) & 1)
    {
        bAmmo = !bAmmo;
        updateDisplay = true;
    }

    if (GetAsyncKeyState(VK_F3) & 1)
    {
        bRecoil = !bRecoil;
        updateDisplay = true;
    }

    if (GetAsyncKeyState(VK_F4) & 1)
    {
        bNoReload = !bNoReload;
        updateDisplay = true;
    }

    if (GetAsyncKeyState(VK_F5) & 1)
    {
        bGrenade = !bGrenade;
        updateDisplay = true;
    }
    if (updateDisplay)
    {
        system("cls");
        std::cout << "==== assaultMe - internal =====\n";
        std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
        std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
        std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
        std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
        std::cout << "[INS] Exit\n";
        std::cout << "=====\n";
    }

    ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
    uintptr_t* localPlayerPtr = (uintptr_t*)(moduleBase + 0x0017E0A8);
```

```

if (localPlayer)
{
    if (bHealth)
    {
        localPlayer->player_health = 1000;
    }

    if (bAmmo)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC73EF), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC73EF), (BYTE*)"\\xFF\\x08", 2);
    }

    if (bNoReload)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC8FC7), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC8FC7), (BYTE*)"\\x01\\x01", 2);
    }
    if (bRecoil)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC2EC3), 5);
    }
    else
    {
        mem::Patch((BYTE*)(moduleBase + 0xC2EC3), (BYTE*)"\\xF3\\x0F\\x11\\x56\\x38", 5);
    }
}
Sleep(5);
}

fclose(f);
FreeConsole();
FreeLibraryAndExitThread(hModule, 0);
return 0;
}

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        // Correct thread creation without CloseHandle
        CreateThread(nullptr, 0, (LPTHREAD_START_ROUTINE)HackThread, hModule, 0, nullptr);
        break;
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:

```

```
case DLL_PROCESS_DETACH:
    break;
}
return TRUE;
}
```

In bRecoil ad esempio metto come address al posto di moduleBase + xxx metto playerpointer + offset non funziona perchè?

Ottima domanda. La ragione principale per cui **non funziona** usare ``playerPointer + offset`` (ovvero qualcosa come ``localPlayer + 0xXYZ``) **nel caso del `bRecoil`**, è che **stai cercando di modificare il codice del gioco (un'istruzione dell'eseguibile in memoria)**, non semplicemente i dati del giocatore.

Vediamo in dettaglio cosa succede:

✓ Quando funziona usare ``playerPointer + offset``

Quando accedi o modifichi **dati** come:

```
cpp
`localPlayer->player_health = 1000;
```

oppure

```
cpp
`localPlayer->grenade_number = 999;
```

In questi casi, ha senso usare ``playerPointer + offset``, perché stai accedendo alla memoria dati dell'entità del giocatore, che cambia a ogni sessione, ed è puntata da ``localPlayer``.

✗ Quando non funziona usare ``playerPointer + offset``

Quando fai cose come:

```
cpp
`mem::Nop((BYTE*)(moduleBase + 0xC2EC3), 5); // No Recoil
```

Stai modificando **le istruzioni macchina (codice eseguibile)** che si trovano nella sezione ``.text`` dell'eseguibile (solitamente in sola lettura). Questo codice non si trova dentro ``localPlayer`` — è fisso rispetto al ``moduleBase``.

Se provi a fare qualcosa tipo:

```
cpp
`mem::Nop((BYTE*)(localPlayer + 0xXYZ), 5); // NON FUNZIONA`
```

Stai dicendo: "vai a scrivere istruzioni macchina in una zona della memoria **riservata ai dati** (il blocco ``ent`` del giocatore)", e questo può:

1. Non avere alcun effetto,
2. Causare un crash,
3. Corrompere dati, ma **non rimuovere il recoil**, perché quello è implementato in codice, non in dati.

In parole semplici:

- ``localPlayer + offset`` → **dati del giocatore**
- ``moduleBase + offset`` → **codice del gioco (istruzioni)**

Vuoi togliere recoil? Ecco il perché del ``moduleBase + 0xC2EC3``

Il recoil è applicato da una **funzione del gioco**. In quell'offset (``0xC2EC3``), c'è un'istruzione come:

```
asm
`movss [esi+38], xmm0`
```

Che modifica il valore del recoil. Se fai un ``NOP`` lì, impedisce al gioco di scrivere il recoil nel valore dei dati del player.

Curiosità (avanzato):

Se volessi davvero fare qualcosa come:

```
cpp
`localPlayer->recoil_value = 0.0f;`
```

Dovresti sapere **esattamente** dove il gioco memorizza temporaneamente il valore del recoil. Ma nella maggior parte dei giochi, questi valori non sono memorizzati permanentemente nel player struct, bensì **calcolati runtime e scritti da codice**.

Fammi sapere se vuoi che ti spiego come trovare quegli offset (``C2EC3``, ``C8FC7``, ecc.) con cheat engine o IDA!



io ho trovato che per modificare il numero di granate del giocatore ha pointer:
ac_client.exe+0017E0A8 però con offset 0x144. come lo posso mettere nel mio codice

Perfetto! Hai trovato correttamente il **pointer al giocatore** e un **offset** (`0x144`) per accedere al campo `grenade_number`.

Nel tuo codice, dato che già hai questa parte:

```
cpp

`ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
```

E il campo `grenade_number` è già definito nella classe `ent`:

```
cpp

`int32_t grenade_number; //0x0144
```

Puoi semplicemente fare:

```
cpp

`localPlayer->grenade_number = 999;
```

oppure, se vuoi farlo direttamente via pointer arithmetic (per imparare):

```
cpp

`*(int*)((uintptr_t)localPlayer + 0x144) = 999;
```

Ma è meglio usare il membro della classe se lo hai già mappato correttamente, come nel tuo `class ent`.

Esempio integrato nel tuo hack:

Aggiungi questo nel loop, dentro l'`if (localPlayer)`:

```
cpp

`if (bGrenade)
{
    localPlayer->grenade_number = 999;
}
```

E per abilitarlo, puoi anche aggiornare il menu:

```
cpp
```

```
`std::cout << "[F5] Grenades : <" << (bGrenade ? "ON " : "OFF") << ">\n";`
```

TL;DR

Hai:

- Base pointer: ``moduleBase + 0x17E0A8``
- Offset del campo: ``0x144``
- Accesso nel codice:

```
cpp
```

```
`localPlayer->grenade_number = 999; // se usi la classe `ent``
```

oppure

```
cpp
```

```
`*(int*)((uintptr_t)localPlayer + 0x144) = 999; // accesso diretto`
```

Vuoi che ti mostri anche come farlo con ``mem::Write`` se stai scrivendo una funzione esterna?



```
#include "stdafx.h"
#include <iostream>
#include "mem.h"
```

```
// Created with ReClass.NET 1.2 by KN4CK3R
```

```
class ent
{
public:
    char pad_0000[4]; //0x0000
    float X; //0x0004
    float Y; //0x0008
    float Z; //0x000C
    char pad_0010[36]; //0x0010
    float lookleft_right; //0x0034
    float lookup_down; //0x0038
    char pad_003C[176]; //0x003C
```



```
int32_t player_health; //0x00EC
int32_t armor_quantity; //0x00F0
char pad_00F4[20]; //0x00F4
int32_t pistol_stored_ammo; //0x0108
char pad_010C[16]; //0x010C
int32_t assault_rifle_stored_ammo; //0x011C
char pad_0120[32]; //0x0120
int32_t assault_rifle_ammo; //0x0140
int32_t grenade_number; //0x0144
char pad_0148[8]; //0x0148
int32_t pistol_shot_reload_delay; //0x0150
char pad_0154[16]; //0x0154
int64_t assault_rifle_shot_reload_delay; //0x0164
char pad_016C[8]; //0x016C
int32_t pistol_total_shots; //0x0174
char pad_0178[16]; //0x0178
int32_t assault_rifle_total_shots; //0x0188
char pad_018C[80]; //0x018C
int32_t number_of_kills; //0x01DC
char pad_01E0[32]; //0x01E0
//char name ? [4]; //0x0200
//char name1[4]; //0x0204
//char name2[4]; //0x0208
char pad_020C[268]; //0x020C
bool is_dead; //0x0318
char pad_0319[75]; //0x0319
class N000002EE* weapon_in_hand; //0x0364
char pad_0368[480]; //0x0368
}; //Size: 0x0548
```

```
class N00000291
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N00000291) == 0x44);
```

```
class N000002EE
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N000002EE) == 0x44);
```

//above is for padding

DWORD WINAPI HackThread(HMODULE hModule)

```
{
// Create Console
AllocConsole();
FILE* f;
freopen_s(&f, "CONOUT$", "w", stdout);

std::cout << "==== assaultMe - internal ====\\n";
std::cout << "[F1] Health Hack : <OFF>\\n";
std::cout << "[F2] Ammo Hack : <OFF>\\n";
std::cout << "[F3] No Recoil : <OFF>\\n";
std::cout << "[F4] No Reload : <OFF>\\n";
std::cout << "[F5] Add Grenades \\n";

std::cout << "=====\\n";
std::cout << "[INS] Exit\\n";

uintptr_t moduleBase = (uintptr_t)GetModuleHandle(L"ac_client.exe");
moduleBase = (uintptr_t)GetModuleHandle(NULL);

bool bHealth = false, bAmmo = false, bRecoil = false, bNoReload = false,
bGrenade = false;

while (true)
{
if (GetAsyncKeyState(VK_INSERT) & 1)
{
break;
}

bool updateDisplay = false;

if (GetAsyncKeyState(VK_F1) & 1)
{
bHealth = !bHealth;
updateDisplay = true;
}

if (GetAsyncKeyState(VK_F2) & 1)
{
bAmmo = !bAmmo;
updateDisplay = true;
}

if (GetAsyncKeyState(VK_F3) & 1)
{
bRecoil = !bRecoil;
updateDisplay = true;
}
}
```

```

if (GetAsyncKeyState(VK_F4) & 1)
{
    bNoReload = !bNoReload;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F5) & 1)
{
    bGrenade = !bGrenade;
    updateDisplay = true;
}
if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
    std::cout << "[F5] Add Grenades : <" << (bGrenade ? "ON " : "OFF") << ">\n";
    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";
}

ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
uintptr_t* localPlayerPtr = (uintptr_t*)(moduleBase + 0x0017E0A8);

if (localPlayer)
{
    if (bHealth)
    {
        localPlayer->player_health = 1000;
    }

    if (bAmmo)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC73EF), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC73EF), (BYTE*)"\\xFF\\x08", 2);
    }

    if (bNoReload)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC8FC7), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC8FC7), (BYTE*)"\\x01\\x01", 2);
    }
    if (bRecoil)

```

```

    {
        mem::Nop((BYTE*)(moduleBase + 0xC2EC3), 5);
    }
    else
    {
        mem::Patch((BYTE*)(moduleBase + 0xC2EC3), (BYTE*)"\\xF3\\x0F\\x11\\x56\\x38", 5);
    }
    if (bGrenade)
    {
        localPlayer->grenade_number = 5;
    }
    else
    {
        localPlayer->grenade_number = 0;
    }
}
Sleep(5);
}

fclose(f);
FreeConsole();
FreeLibraryAndExitThread(hModule, 0);
return 0;
}

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
            // Correct thread creation without CloseHandle
            CreateThread(nullptr, 0, (LPTHREAD_START_ROUTINE)HackThread, hModule, 0, nullptr);
            break;
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}
io ho messo 5 ma sembrano infinite

```

Sì, è del tutto normale! 😊 Quando imposti il numero di granate a `5` ad ogni ciclo (`Sleep(5)` → 200 volte al secondo), stai praticamente facendo un **refresh continuo** del numero di granate. Quindi anche se ne lanci una, nel prossimo frame il valore torna a `5`. Risultato: **granate infinite** 🚀

✅ Vuoi 5 granate fisse?

Basta impostarlo **una volta sola** (non in ogni ciclo), ad esempio:

```
cpp

`if (bGrenade)
{
    if (localPlayer->grenade_number < 5)
        localPlayer->grenade_number = 5;
}
```

Oppure un'alternativa:

```
cpp

`static bool alreadyGiven = false;
if (bGrenade && !alreadyGiven)
{
    localPlayer->grenade_number = 5;
    alreadyGiven = true;
}
else if (!bGrenade)
{
    alreadyGiven = false;
}
```

✓ Vuoi granate infinite?

Allora **lascia pure com'è ora**:

```
cpp

`if (bGrenade)
{
    localPlayer->grenade_number = 5;
}
else
{
    localPlayer->grenade_number = 0;
}
```

In pratica, ti stai autorifornendo in ogni singolo frame. È un trucchetto classico 😎

Fammi sapere se vuoi anche:

- il numero di granate visualizzato sullo schermo (in overlay)
- rendere la quantità **augmentabile con un tasto**
- oppure se vuoi creare un cheat menu interattivo in-game.



io non voglio che siano infinite

Perfetto! Allora vediamo subito come **evitare che le granate siano infinite** e far sì che vengano **aggiunte solo una volta** quando premi F5.

🔧 Modifica il blocco ``if (bGrenade)`` così:

Sostituisci questo:

```
cpp

`if (bGrenade)
{
    localPlayer->grenade_number = 5;
}
else
{
    localPlayer->grenade_number = 0;
}
```

Con questo:

```
cpp

`static bool grenadeGiven = false;

if (bGrenade && !grenadeGiven)
{
    localPlayer->grenade_number += 5; // Aggiunge 5 granate
    grenadeGiven = true;
}
else if (!bGrenade)
{
    grenadeGiven = false; // Pronto per il prossimo F5
}
```

✅ Cosa succede adesso?

- Premi ``F5`` una volta → aggiunge 5 granate.
- Se premi ``F5`` di nuovo, ne aggiunge altre 5.
- Le granate non diventano infinite: se le usi, calano.
- Non vengono più **reset ogni frame**, solo aggiunte quando premi.

Fammi sapere se vuoi un numero **preciso** invece di fare ``+= 5``, oppure un piccolo **menu** con quantità configurabile.



```
#include "stdafx.h"
#include <iostream>
#include "mem.h"
```

```
// Created with ReClass.NET 1.2 by KN4CK3R
```

```
class ent
{
public:
    char pad_0000[4]; //0x0000
    float X; //0x0004
    float Y; //0x0008
    float Z; //0x000C
    char pad_0010[36]; //0x0010
    float lookleft_right; //0x0034
    float lookup_down; //0x0038
    char pad_003C[176]; //0x003C
    int32_t player_health; //0x00EC
    int32_t armor_quantity; //0x00F0
    char pad_00F4[20]; //0x00F4
    int32_t pistol_stored_ammo; //0x0108
    char pad_010C[16]; //0x010C
    int32_t assault_rifle_stored_ammo; //0x011C
    char pad_0120[32]; //0x0120
    int32_t assault_rifle_ammo; //0x0140
    int32_t grenade_number; //0x0144
    char pad_0148[8]; //0x0148
    int32_t pistol_shot_reload_delay; //0x0150
    char pad_0154[16]; //0x0154
    int64_t assault_rifle_shot_reload_delay; //0x0164
    char pad_016C[8]; //0x016C
    int32_t pistol_total_shots; //0x0174
    char pad_0178[16]; //0x0178
    int32_t assault_rifle_total_shots; //0x0188
    char pad_018C[80]; //0x018C
    int32_t number_of_kills; //0x01DC
    char pad_01E0[32]; //0x01E0
    //char name ? [4]; //0x0200
    //char name1[4]; //0x0204
    //char name2[4]; //0x0208
    char pad_020C[268]; //0x020C
    bool is_dead; //0x0318
    char pad_0319[75]; //0x0319
    class N000002EE* weapon_in_hand; //0x0364
    char pad_0368[480]; //0x0368
}; //Size: 0x0548
```

```
class N00000291
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N00000291) == 0x44);
```

```
class N000002EE
```

```
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N0000002EE) == 0x44);


//above is for padding


DWORD WINAPI HackThread(HMODULE hModule)
{
    // Create Console
    AllocConsole();
    FILE* f;
    freopen_s(&f, "CONOUT$", "w", stdout);

    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <OFF>\n";
    std::cout << "[F2] Ammo Hack : <OFF>\n";
    std::cout << "[F3] No Recoil : <OFF>\n";
    std::cout << "[F4] No Reload : <OFF>\n";
    std::cout << "[F5] Add Grenades \n";

    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";

    uintptr_t moduleBase = (uintptr_t)GetModuleHandle(L"ac_client.exe");
    moduleBase = (uintptr_t)GetModuleHandle(NULL);

    bool bHealth = false, bAmmo = false, bRecoil = false, bNoReload = false,
        bGrenade = false;

    //random declarations
    static bool grenadeGiven = false;

    while (true)
    {
        if (GetAsyncKeyState(VK_INSERT) & 1)
        {
            break;
        }

        bool updateDisplay = false;

        if (GetAsyncKeyState(VK_F1) & 1)
        {
            bHealth = !bHealth;
            updateDisplay = true;
        }
    }
}
```



```

}

if (GetAsyncKeyState(VK_F2) & 1)
{
    bAmmo = !bAmmo;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F3) & 1)
{
    bRecoil = !bRecoil;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F4) & 1)
{
    bNoReload = !bNoReload;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F5) & 1)
{
    bGrenade = !bGrenade;
    updateDisplay = true;
}

if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
    std::cout << "[F5] Add 10 Grenades : <" << (grenade_number) << ">\n";
    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";
}

ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
uintptr_t* localPlayerPtr = (uintptr_t*)(moduleBase + 0x0017E0A8);

if (localPlayer)
{
    if (bHealth)
    {
        localPlayer->player_health = 1000;
    }

    if (bAmmo)
    {

```

```

        mem::Nop((BYTE*)(moduleBase + 0xC73EF), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC73EF), (BYTE*)"\\xFF\\x08", 2);
    }

    if (bNoReload)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC8FC7), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC8FC7), (BYTE*)"\\x01\\x01", 2);
    }
    if (bRecoil)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC2EC3), 5);
    }
    else
    {
        mem::Patch((BYTE*)(moduleBase + 0xC2EC3), (BYTE*)"\\xF3\\x0F\\x11\\x56\\x38", 5);
    }
    if (bGrenade && !grenadeGiven)
    {
        localPlayer->grenade_number += 10;
        grenadeGiven = true;
    }
    else if (!bGrenade)
    {
        grenadeGiven = false;
    }
}
Sleep(5);
}

fclose(f);
FreeConsole();
FreeLibraryAndExitThread(hModule, 0);
return 0;
}

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        // Correct thread creation without CloseHandle
        CreateThread(nullptr, 0, (LPTHREAD_START_ROUTINE)HackThread, hModule, 0, nullptr);
        break;
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:

```

```
break;
}
return TRUE;
}
```

Perfetto, grazie per aver condiviso tutto il codice! Ti sei mosso molto bene 👍

Ora vediamo insieme **perché il numero di granate è ancora potenzialmente "infinito" o incostante nel display**, e come sistemare tutto in modo **preciso e pulito**:

🔧 PROBLEMI ATTUALI

1. Variabile ``grenade_number`` non definita globalmente → stai stampando:

```
cpp

`std::cout << "[F5] Add 10 Grenades : <" << (grenade_number) << ">\n";`
```

Ma ``grenade_number`` non esiste come variabile globale → questo può dare risultati non previsti o errore in compilazione.

2. Il toggle ``bGrenade`` funziona come interruttore (ON/OFF) → ma tu vuoi solo **un colpo singolo di 10 granate** ogni volta che premi F5.

✅ SOLUZIONE COMPLETA

1. ✅ Sostituisci il blocco ``F5`` con questo:

```
cpp

`if (GetAsyncKeyState(VK_F5) & 1)
{
    if (localPlayer)
    {
        localPlayer->grenade_number += 10;
        updateDisplay = true;
    }
}
```

2. ✅ Sostituisci la stampa nel menu:

```
cpp

`std::cout << "[F5] Add 10 Grenades : <"
    << (localPlayer ? localPlayer->grenade_number : 0) << ">\n";`
```

🔄 COSA CAMBIA?

- Non usi più `bGrenade` o `grenadeGiven`: non servono.
- Ogni volta che premi `F5`, aggiungi **esattamente 10 granate**.
- Non ci sono toggle ON/OFF, è solo un colpo singolo.
- Lo stato attuale delle granate viene **stampato sempre correttamente**.

BONUS: comportamento desiderato

-  Granate **non infinite**.
-  Premi F5 → aggiungi 10.
-  Le consumi in gioco → calano.
-  Premendo più volte F5 → puoi ricaricare.

Se vuoi anche un "reset a zero" con un altro tasto (es. `F6`), basta aggiungerlo con:

```
cpp

`if (GetAsyncKeyState(VK_F6) & 1)
{
    if (localPlayer)
    {
        localPlayer->grenade_number = 0;
        updateDisplay = true;
    }
}
```

Fammi sapere se vuoi che il numero massimo sia limitato (es. massimo 99), o altre funzioni da aggiungere!



dove metto la stampa

Ottima domanda! 🕶️

La stampa che mostra quante granate hai la metti nel **blocco** `if (updateDisplay)`, dove stampi il menu aggiornato.

ECCO COME MODIFICARLO

Sostituisci questo blocco nel tuo codice:

```
cpp

`if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
```

```
std::cout << "[F5] Add 10 Grenades : <" << (grenade_number) << ">\n";
std::cout << "=====\n";
std::cout << "[INS] Exit\n";
}
```

CON QUESTO:

```
cpp

`if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";

    ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
    int grenades = (localPlayer ? localPlayer->grenade_number : 0);

    std::cout << "[F5] Add 10 Grenades : <" << grenades << ">\n";
    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";
}
```

✓ RISULTATO

- Mostra sempre il **numero attuale** di granate.
- Niente variabili inutili.
- Facile da estendere (puoi farlo anche per colpi, vita, ecc.).

Fammi sapere se vuoi anche un mini HUD oppure una cosa più visual tipo `██████ 6/10`.



```
#include "stdafx.h"
#include <iostream>
#include "mem.h"
```

```
// Created with ReClass.NET 1.2 by KN4CK3R
```

```
class ent
{
public:
    char pad_0000[4]; //0x0000
    float X; //0x0004
    float Y; //0x0008
```

```
float Z; //0x000C
char pad_0010[36]; //0x0010
float lookleft_right; //0x0034
float lookup_down; //0x0038
char pad_003C[176]; //0x003C
int32_t player_health; //0x00EC
int32_t armor_quantity; //0x00F0
char pad_00F4[20]; //0x00F4
int32_t pistol_stored_ammo; //0x0108
char pad_010C[16]; //0x010C
int32_t assault_rifle_stored_ammo; //0x011C
char pad_0120[32]; //0x0120
int32_t assault_rifle_ammo; //0x0140
int32_t grenade_number; //0x0144
char pad_0148[8]; //0x0148
int32_t pistol_shot_reload_delay; //0x0150
char pad_0154[16]; //0x0154
int64_t assault_rifle_shot_reload_delay; //0x0164
char pad_016C[8]; //0x016C
int32_t pistol_total_shots; //0x0174
char pad_0178[16]; //0x0178
int32_t assault_rifle_total_shots; //0x0188
char pad_018C[80]; //0x018C
int32_t number_of_kills; //0x01DC
char pad_01E0[32]; //0x01E0
//char name ? [4]; //0x0200
//char name1[4]; //0x0204
//char name2[4]; //0x0208
char pad_020C[268]; //0x020C
bool is_dead; //0x0318
char pad_0319[75]; //0x0319
class N000002EE* weapon_in_hand; //0x0364
char pad_0368[480]; //0x0368
}; //Size: 0x0548
```

```
class N00000291
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N00000291) == 0x44);
```

```
class N000002EE
{
public:
    char pad_0000[68]; //0x0000
}; //Size: 0x0044
static_assert(sizeof(N000002EE) == 0x44);
```

```
//above is for padding
```

```
DWORD WINAPI HackThread(HMODULE hModule)
{
    // Create Console
    AllocConsole();
    FILE* f;
    freopen_s(&f, "CONOUT$", "w", stdout);

    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <OFF>\n";
    std::cout << "[F2] Ammo Hack : <OFF>\n";
    std::cout << "[F3] No Recoil : <OFF>\n";
    std::cout << "[F4] No Reload : <OFF>\n";
    std::cout << "[F5] Add 10 Grenades : <" << "0" << ">\n";

    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";

    uintptr_t moduleBase = (uintptr_t)GetModuleHandle(L"ac_client.exe");
    moduleBase = (uintptr_t)GetModuleHandle(NULL);

    bool bHealth = false, bAmmo = false, bRecoil = false, bNoReload = false,
        bGrenade = false;

    //random declarations
    static bool grenadeGiven = false;

    while (true)
    {
        ent* localPlayer = *(ent**)(moduleBase + 0x0017E0A8);
        uintptr_t* localPlayerPtr = (uintptr_t*)(moduleBase + 0x0017E0A8);

        if (GetAsyncKeyState(VK_INSERT) & 1)
        {
            break;
        }

        bool updateDisplay = false;

        if (GetAsyncKeyState(VK_F1) & 1)
        {
            bHealth = !bHealth;
            updateDisplay = true;
        }

        if (GetAsyncKeyState(VK_F2) & 1)
        {

```

```

    bAmmo = !bAmmo;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F3) & 1)
{
    bRecoil = !bRecoil;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F4) & 1)
{
    bNoReload = !bNoReload;
    updateDisplay = true;
}

if (GetAsyncKeyState(VK_F5) & 1)
{
    bGrenade = !bGrenade;
    updateDisplay = true;
}

if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
    std::cout << "[F5] Add 10 Grenades : <" << (localPlayer->grenade_number) << ">\n";
    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";
}

if (localPlayer)
{
    if (bHealth)
    {
        localPlayer->player_health = 1000;
    }

    if (bAmmo)
    {
        mem::Nop((BYTE*)(moduleBase + 0xC73EF), 2);
    }
    else {
        mem::Patch((BYTE*)(moduleBase + 0xC73EF), (BYTE*)"\\xFF\\x08", 2);
    }

    if (bNoReload)

```



```

{
    mem::Nop((BYTE*)(moduleBase + 0xC8FC7), 2);
}
else {
    mem::Patch((BYTE*)(moduleBase + 0xC8FC7), (BYTE*)"0101", 2);
}
if (bRecoil)
{
    mem::Nop((BYTE*)(moduleBase + 0xC2EC3), 5);
}
else
{
    mem::Patch((BYTE*)(moduleBase + 0xC2EC3), (BYTE*)"F30F115638", 5);
}
if (bGrenade && !grenadeGiven)
{
    localPlayer->grenade_number += 10;
    grenadeGiven = true;
}
else if (!bGrenade)
{
    grenadeGiven = false;
}
}
Sleep(5);
}

fclose(f);
FreeConsole();
FreeLibraryAndExitThread(hModule, 0);
return 0;
}

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call, LPVOID lpReserved)
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        // Correct thread creation without CloseHandle
        CreateThread(nullptr, 0, (LPTHREAD_START_ROUTINE)HackThread, hModule, 0, nullptr);
        break;
    case DLL_THREAD_ATTACH:
    case DLL_THREAD_DETACH:
    case DLL_PROCESS_DETACH:
        break;
    }
    return TRUE;
}
counter doesnt work

```

Grazie per aver incollato l'intero codice! 🔍

Hai fatto un ottimo lavoro, il problema ora è piuttosto sottile: la **logica del contatore delle granate** (``grenade_number``) **non aggiorna il valore correttamente nel menu** perché ``bGrenade`` viene attivato/disattivato con ``F5``, ma ``grenadeGiven`` impedisce che venga sommato più di una volta... e questo avviene **solo quando premi F5**.

💣 Perché il contatore "non funziona"?

Esempio scenario attuale:

1. Premi F5 → ``bGrenade`` diventa ``true`` e ``grenadeGiven`` è ``false`` → Aggiunge 10 granate e ``grenadeGiven`` diventa ``true``.
2. La stampa viene aggiornata.
3. Premi F5 di nuovo → ``bGrenade`` diventa ``false``, ``grenadeGiven`` torna ``false``, ma **nessuna granata viene aggiunta finché non premi di nuovo F5**.

⚠️ Ma intanto nel menu appare il contatore vecchio (es. ancora 0 se non è entrato bene nel ramo).

✅ Soluzione semplice ed efficace

Sposta l'**aggiunta delle granate fuori dal blocco** ``if (bGrenade)`` e gestiscila direttamente quando premi ``F5``.

🔧 CAMBIA QUESTO:

```
cpp
`if (GetAsyncKeyState(VK_F5) & 1)
{
    bGrenade = !bGrenade;
    updateDisplay = true;
}
```

E anche **questa parte**:

```
cpp
`if (bGrenade && !grenadeGiven)
{
    localPlayer->grenade_number += 10;
    grenadeGiven = true;
}
else if (!bGrenade)
{
    grenadeGiven = false;
}
```

🔄 CON QUESTO:

```
cpp

`if (GetAsyncKeyState(VK_F5) & 1)
{
    if (localPlayer)
    {
        localPlayer->grenade_number += 10;
    }
    updateDisplay = true;
}
```

✗ Inoltre: puoi rimuovere queste variabili inutili:

```
cpp

`bGrenade = false;
grenadeGiven = false;
```

✓ Risultato

- Premi F5 → aggiunge subito 10 granate.
- Il numero mostrato è sempre corretto.
- Nessun toggle, solo azione diretta.

Se vuoi che **si attivi un cheat continuo di "granate infinite"**, fammi sapere e te lo faccio con un toggle e auto-refill 😊.



no ma funziona tutto solo che quando schiaccio f5 la prima volta non va a 10 ma rimane 0, invece la seconda funziona

Perfetto, grazie per la conferma! 🙌

Ora che sappiamo che **tutto funziona**, ma **la prima pressione di `F5` non aggiorna il contatore**, ti spiego esattamente perché succede — e come risolverlo senza riscrivere la logica.

🧠 Il motivo preciso:

Nel tuo codice, quando premi `F5`, fai **due cose separate**:

1. Cambi `bGrenade` da `false` a `true`.
2. Solo **nel ciclo successivo** (`if (bGrenade && !grenadeGiven)`) aggiungi le granate.

🔄 Ma il menu (`system("cls")` + `std::cout`) lo stampi **subito dopo aver cambiato `bGrenade`**, **prima** che l'effettiva aggiunta delle granate venga fatta al ciclo seguente.

🔧 Soluzione elegante: forza la stampa dopo aver aggiornato le granate

Sposta la stampa del menu **dopo la parte dove modifichi** ``localPlayer->grenade_number``.

✅ Modifica suggerita:

SPOSTA QUESTO BLOCCO:

```
cpp

`if (updateDisplay)
{
    system("cls");
    std::cout << "==== assaultMe - internal =====\n";
    std::cout << "[F1] Health Hack : <" << (bHealth ? "ON " : "OFF") << ">\n";
    std::cout << "[F2] Ammo Hack : <" << (bAmmo ? "ON " : "OFF") << ">\n";
    std::cout << "[F3] No Recoil : <" << (bRecoil ? "ON " : "OFF") << ">\n";
    std::cout << "[F4] No Reload : <" << (bNoReload ? "ON " : "OFF") << ">\n";
    std::cout << "[F5] Add 10 Grenades : <" << (localPlayer->grenade_number) << ">\n";
    std::cout << "=====\n";
    std::cout << "[INS] Exit\n";
}
`
```

METTILO dopo questo blocco:

```
cpp

`if (bGrenade && !grenadeGiven)
{
    localPlayer->grenade_number += 10;
    grenadeGiven = true;
}
else if (!bGrenade)
{
    grenadeGiven = false;
}
`
```

🟢 Così facendo:

- La **stampa avviene dopo** l'aggiunta effettiva delle granate.
- Quindi, la **prima pressione di `F5` mostrerà direttamente `10`, non `0`.**

Fammi sapere se vuoi anche colorare il testo tipo verde/rosso per ON/OFF con ``SetConsoleTextAttribute``



posso impostare una dimensione e posizione default per la console

Sì, puoi **impostare dimensione e posizione predefinita** della console Windows nel tuo codice con le API della Win32. Ti spiego subito come farlo 🙌

✓ 1. Imposta la dimensione della finestra della console

Per esempio: 80 colonne × 25 righe

```
cpp

`HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

SMALL_RECT rect = {0, 0, 79, 24}; // (left, top, right, bottom)
SetConsoleWindowInfo(hConsole, TRUE, &rect);
```

💡 Tip:

Se non funziona da solo, prima **imposta il buffer** grande abbastanza:

```
cpp

`COORD bufferSize = {80, 25};
SetConsoleScreenBufferSize(hConsole, bufferSize);
```

✓ 2. Imposta la posizione della finestra

Per esempio: spostala a (100, 100) sullo schermo:

```
cpp

`HWND hwnd = GetConsoleWindow();
MoveWindow(hwnd, 100, 100, 800, 600, TRUE); // x, y, width, height, repaint
```

Dove lo metto?

Metti tutto questo all'inizio del tuo ``HackThread``, **dopo** ``AllocConsole()`` ma **prima** di stampare roba.

```
cpp

`AllocConsole();
FILE* f;
freopen_s(&f, "CONOUT$", "w", stdout);

// SETUP: dimensione e posizione console
HWND hwnd = GetConsoleWindow();
MoveWindow(hwnd, 100, 100, 800, 600, TRUE); // posizione + dimensione (in pixel)

HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
COORD bufferSize = {80, 25};
SetConsoleScreenBufferSize(hConsole, bufferSize);

SMALL_RECT rect = {0, 0, 79, 24};
```

```
SetConsoleWindowInfo(hConsole, TRUE, &rect);
```

Risultato:

- Finestra si apre **in alto a sinistra** (o dove vuoi tu).
- Con **larghezza 80** e **altezza 25** caratteri.
- Visivamente ordinata e controllata 

Fammi sapere se vuoi anche **nascondere la barra del titolo** o **disabilitare il resize** 🐼